

Lesson 1 Structured Note: Introduction to Stacks and the Stack ADT

0. Current Position in the Course

You are starting **Lesson 1** of the stack curriculum.

This lesson belongs to:

Phase 1: Foundations and Array Stack

At this point, the main goal is not to write heavy code yet. The goal is to understand the **idea of a stack** clearly.

Lesson 1 Goal

By the end of this lesson, you should understand:

- What a stack is
 - What LIFO means
 - Why stack operations happen only at the top
 - What `push`, `pop`, `peek`, `stackTop`, `isEmpty`, and `isFull` do
 - How the `top` value changes during stack operations
 - What a stack ADT means
 - Why basic stack operations are usually constant time: $O(1)$
-

1. What Is a Stack?

A **stack** is a linear data structure.

A linear data structure stores items in a sequence, one after another.

Examples of linear data structures include:

- Arrays
- Linked lists
- Stacks
- Queues

A stack has one special rule:

Last In, First Out

This is also called:

LIFO

2. Meaning of LIFO

LIFO = Last In, First Out

This means:

The last item inserted into the stack is the first item removed from the stack.

For example, suppose we insert these values:

```
push(10)
push(20)
push(30)
```

The stack looks like this:

```
Top -> 30
       20
       10
```

The last inserted value is **30**.

So, if we remove one value using `pop()`, the value removed will be:

```
30
```

The value **10** was inserted first, but it cannot be removed first because it is at the bottom.

3. Simple Real-Life Examples of Stacks

Example 1: Stack of Plates

Imagine plates placed one above another.

```
Top -> Plate 3
      Plate 2
      Plate 1
```

The last plate placed is **Plate 3**.

When you remove a plate, you remove **Plate 3** first.

This follows LIFO.

Example 2: Tennis Balls in a Tube

Imagine putting tennis balls into a narrow tube.

```
Ball 1 goes in first
Ball 2 goes in second
Ball 3 goes in third
```

The last ball inserted is **Ball 3**.

When you remove a ball, **Ball 3** comes out first.

This also follows LIFO.

Example 3: Browser Back Button

Suppose you visit websites in this order:

```
Google
YouTube
GitHub
```

The most recent page is **GitHub**.

When you press the back button, you return to:

```
YouTube
```

The browser removes the latest page first.

This is similar to stack behavior.

4. The Most Important Part of a Stack: `top`

In a stack, we only work with one end.

That end is called the **top**.

Example:

```
Top -> 30
       20
       10
```

Here:

- `30` is the top element
- `20` is in the middle
- `10` is at the bottom

In a normal stack, we do not insert or delete from the middle.

So this is allowed:

```
pop()      removes 30
push(40)    adds 40 above 30
```

But this is not allowed in a standard stack:

```
Remove 20 directly from the middle
```

That would break the stack rule.

Important Idea

A stack is not just any list of values.

A stack is a list with a strict access rule:

Insert and delete only from the top.

5. Basic Stack Operations

A stack usually provides a fixed set of operations.

These operations are:

Operation	Simple Meaning
<code>push(x)</code>	Insert <code>x</code> at the top
<code>pop()</code>	Remove and return the top element
<code>peek(pos)</code>	Read an element at a position from the top
<code>stackTop()</code>	Read the current top element
<code>isEmpty()</code>	Check whether the stack is empty
<code>isFull()</code>	Check whether the stack is full

At this stage, focus on what each operation does, not the full code.

6. `push(x)` Operation

Meaning

`push(x)` means:

Insert value `x` at the top of the stack.

Example

Start with an empty stack:

```
top = -1
```

Now perform:

```
push(10)
```

The stack becomes:

```
Top -> 10
```

Now perform:

```
push(16)
```

The stack becomes:

```
Top -> 16  
      10
```

Now perform:

```
push(20)
```

The stack becomes:

```
Top -> 20  
      16  
      10
```

Key Point

Every new value is inserted above the previous top value.

7. `pop()` Operation

Meaning

`pop()` means:

Remove and return the top element from the stack.

Example

Current stack:

```
Top -> 20
      16
      10
```

Now call:

```
pop()
```

The top value is 20, so pop() returns:

```
20
```

After removing 20, the stack becomes:

```
Top -> 16
      10
```

Key Point

pop() changes the stack because it removes the top element.

8. stackTop() Operation

Meaning

stackTop() means:

Read the top element without removing it.

Example

Current stack:

```
Top -> 16
      10
```

Now call:

```
stackTop()
```

It returns:

```
16
```

But the stack does not change.

It remains:

```
Top -> 16
      10
```

Difference Between `pop()` and `stackTop()`

Operation	Returns Top Value?	Removes Top Value?
<code>pop()</code>	Yes	Yes
<code>stackTop()</code>	Yes	No

9. `peek(pos)` Operation

Meaning

`peek(pos)` means:

Read a value at a given position from the top without deleting it.

The position is counted from the top.

Example:

```
Top -> 30  position 1
      25  position 2
      20  position 3
      16  position 4
      10  position 5
```


So:

```
peek(1) returns 30  
peek(2) returns 25  
peek(3) returns 20
```

Key Point

`peek()` only reads a value.

It does not remove anything.

10. `isEmpty()` Operation

Meaning

`isEmpty()` checks whether the stack has no elements.

For an array-based stack, an empty stack is usually represented by:

```
top = -1
```

So:

```
if top == -1, the stack is empty
```

Example:

```
top = -1
```

This means:

```
The stack has no active elements.
```

11. `isFull()` Operation

Meaning

`isFull()` checks whether the stack has reached its maximum capacity.

In an array-based stack, the stack has a fixed size.

If the array size is `5`, the valid indices are:

```
0, 1, 2, 3, 4
```

The stack is full when:

```
top == 4
```

More generally:

```
top == size - 1
```

Example:

If:

```
size = 5
```

Then the stack is full when:

```
top == 5 - 1  
top == 4
```

12. Array View of a Stack

A stack can be implemented using an array.

Suppose the stack size is `5`.

The array indices are:

```
Index:  0   1   2   3   4
Value: [ ] [ ] [ ] [ ] [ ]
```

When the stack is empty:

```
top = -1
```

Why is `top = -1` used?

Because no valid array position is being used yet.

Array indices start from `0`.

So `-1` means:

```
No element exists in the stack.
```

13. How `top` Changes During Push

Start:

```
Index:  0   1   2   3   4
Value: [ ] [ ] [ ] [ ] [ ]
top = -1
```

After:

```
push(10)
```

The stack becomes:

```
Index:  0   1   2   3   4
Value: [10] [ ] [ ] [ ] [ ]
top = 0
```

After:

```
push(16)
```

The stack becomes:

```
Index:  0    1    2    3    4
Value: [10] [16] [ ] [ ] [ ]
top = 1
```

After:

```
push(20)
```

The stack becomes:

```
Index:  0    1    2    3    4
Value: [10] [16] [20] [ ] [ ]
top = 2
```

Key Point

Each successful `push()` increases `top` by `1`.

14. Required Dry Run 1: Push 10, 16, 20, 25, 30

Given

```
size = 5
top = -1
```

We perform:

```
push(10)
push(16)
push(20)
```

```
push(25)
push(30)
```

Step-by-Step Trace

Step	Operation	top Before	top After	Stack Contents
0	Start	-1	-1	Empty
1	push(10)	-1	0	10
2	push(16)	0	1	10, 16
3	push(20)	1	2	10, 16, 20
4	push(25)	2	3	10, 16, 20, 25
5	push(30)	3	4	10, 16, 20, 25, 30

Final Array View

```
Index:  0    1    2    3    4
Value: [10] [16] [20] [25] [30]
top = 4
```

Final Logical Stack View

```
Top -> 30
      25
      20
      16
      10
```

Important Result

After pushing five values into a stack of size 5:

```
top = 4
```

The top value is:

30

15. Required Dry Run 2: Pop Once

Start with this stack:

```
Index:  0    1    2    3    4
Value: [10] [16] [20] [25] [30]
top = 4
```

Logical stack:

```
Top -> 30
      25
      20
      16
      10
```

Now perform:

```
pop()
```

Steps

1. Read the value at S[top]
2. S[4] is 30
3. Save 30 as the return value
4. Decrease top from 4 to 3
5. Return 30

Result

```
pop() returns 30
```

After the pop:

```
top = 3
```

The logical stack becomes:

```
Top -> 25
      20
      16
      10
```

Important Memory Note

The value `30` may still physically exist in the array memory.

But it is not logically part of the stack anymore because `top = 3`.

So index `4` is ignored.

Active stack:

```
10, 16, 20, 25
```

Ignored value:

```
30 at index 4
```

16. Required Dry Run 3: `peek(3)` When `top = 3`

Current stack:

```
Index: 0    1    2    3
Value: [10] [16] [20] [25]
top = 3
```

Logical view:

```
Top -> 25    position 1
      20    position 2
```

```
16    position 3
10    position 4
```

We want:

```
peek(3)
```

This means:

Return the third element from the top.

Formula

```
index = top - position + 1
```

Substitute the values:

```
index = 3 - 3 + 1
index = 1
```

So:

```
peek(3) returns S[1]
```

Since:

```
S[1] = 16
```

The answer is:

```
peek(3) returns 16
```

17. Stack ADT

Meaning of ADT

ADT means:

Abstract Data Type

An ADT describes what operations are possible without focusing first on how those operations are implemented.

For a stack, the ADT tells us that these operations exist:

```
push(x)
pop()
peek(pos)
stackTop()
isEmpty()
isFull()
```

Why ADT Is Useful

The user of the stack does not need to know whether the stack is built using:

- An array
- A linked list
- A C structure
- A C++ class
- Dynamic memory

The user only needs to know how to use the stack operations.

Important Idea

The stack concept stays the same even if the implementation changes.

For example:

```
Array stack      -> still follows LIFO
Linked-list stack -> still follows LIFO
C++ class stack  -> still follows LIFO
```

The implementation can change, but the behavior remains the same.

18. System Stack vs Programmer Stack

There are two stack ideas you should separate.

18.1 System Stack

The **system stack** is managed automatically by the computer.

It is used when functions call other functions.

Example:

```
main() calls A()  
A() calls B()  
B() calls C()
```

The most recent function call must finish first.

So the finishing order is:

```
C finishes first  
B finishes next  
A finishes next  
main finishes last
```

This follows LIFO.

System Stack and Recursion

Recursion uses the system stack.

When a function calls itself, each function call is stored on the system stack.

The last recursive call finishes first.

That is why recursion is connected to stack behavior.

18.2 Programmer Stack

A **programmer stack** is a stack that you create yourself in your program.

Example:

```
push(10)
push(20)
pop()
```

Here, you are directly controlling the stack operations.

Main Difference

Type	Who Manages It?	Example
System stack	Computer/compiler/runtime	Function calls and recursion
Programmer stack	Programmer	Stack data structure in code

19. Recursion vs Iteration and Explicit Stacks

Recursion often uses the system stack automatically.

Sometimes, recursive logic can be rewritten using iteration.

When we rewrite recursion using iteration, we may need to create our own stack.

That stack is called an **explicit stack**.

Simple Meaning

- Recursion: stack is handled automatically by the system
- Iteration with stack: programmer creates and controls the stack manually

Example idea:

```
Recursive solution -> uses system stack
Iterative solution -> may use programmer-created stack
```

20. Time Complexity of Basic Stack Operations

Most basic stack operations are $O(1)$.

$O(1)$ means:

Constant time

This means the operation takes roughly the same amount of work no matter how many elements are in the stack.

Complexity Table

Operation	Time Complexity	Reason
<code>push(x)</code>	<code>O(1)</code>	Inserts only at the top
<code>pop()</code>	<code>O(1)</code>	Removes only from the top
<code>stackTop()</code>	<code>O(1)</code>	Reads only the top
<code>isEmpty()</code>	<code>O(1)</code>	Checks the <code>top</code> value
<code>isFull()</code>	<code>O(1)</code>	Checks if <code>top == size - 1</code>

Why These Are `O(1)`

The operation does not scan the whole stack.

For example, `pop()` already knows where the top is.

It simply removes the top value.

It does not search through all elements.

21. Space Complexity of Basic Operations

The individual operations such as `push`, `pop`, and `stackTop` usually use only a fixed number of extra variables.

So their auxiliary space is:

`O(1)`

This means they do not need extra memory that grows with the stack size.

However, the stack itself needs storage.

For example, an array stack of size `5` needs space for `5` elements.

But a single operation like `pop()` only needs a small fixed amount of extra memory.

22. Stack Implementation Choices

A stack can be implemented in different ways.

Common implementations are:

- Array-based stack
- Linked-list-based stack

Array Stack

An array stack uses an array to store values.

Example:

```
Index:  0    1    2
Value: [10] [20] [30]
```

It usually has:

```
size
array storage
top index
```

Linked-List Stack

A linked-list stack uses nodes connected by pointers.

Example:

```
Top -> 30 -> 20 -> 10 -> NULL
```

Important Point

The implementation can change, but the stack rule does not change.

Both array stack and linked-list stack follow:

Last In, First Out

23. Empty State in an Array Stack

In an array stack, the empty state is usually:

`top = -1`

This means:

There is no active element in the stack.

It does not mean array index `-1` exists.

Array index `-1` is invalid.

It is just a marker value.

24. Full State in an Array Stack

If the stack is stored in an array, it can become full.

If:

`size = 5`

Then the valid array indices are:

`0, 1, 2, 3, 4`

The stack is full when:

`top = 4`

General formula:

```
top == size - 1
```

When the stack is full, another `push()` would cause overflow.

25. Overflow and Underflow

Overflow

Overflow happens when you try to push into a full stack.

Example:

```
size = 5  
top = 4
```

The stack is already full.

If you do:

```
push(40)
```

That causes overflow.

Underflow

Underflow happens when you try to pop from an empty stack.

Example:

```
top = -1
```

The stack is empty.

If you do:

```
pop()
```

That causes underflow.

26. Common Beginner Mistakes

Mistake 1: Thinking a Stack Is Just Any List

A stack is not just any list.

A stack is a list with a rule:

```
Last In, First Out
```

Mistake 2: Removing from the Middle

This is not standard stack behavior:

```
Remove the middle element directly
```

A stack only allows insertion and deletion at the top.

Mistake 3: Confusing Physical Memory with Logical Stack Content

After `pop()`, a value may still remain in memory.

But if `top` has moved down, that value is no longer part of the stack.

Example:

```
Index:  0    1    2    3    4
Value: [10] [16] [20] [25] [30]
top = 3
```

Here, `30` still appears at index `4`.

But it is ignored because:

```
top = 3
```

The active stack is only:

10, 16, 20, 25

Mistake 4: Thinking `peek()` Removes Data

`peek()` does not remove data.

It only reads.

The operation that removes data is:

`pop()`

Mistake 5: Forgetting That Position in `peek(pos)` Starts from the Top

In `peek(pos)`, position `1` means the top element.

Example:

```
Top -> 30   position 1
        25   position 2
        20   position 3
```

So:

```
peek(1) = 30
peek(2) = 25
peek(3) = 20
```

27. Mini Practice Trace

Trace this sequence:

```
push(7)
push(9)
pop()
```

```
push(11)
stackTop()
```

Step-by-Step Answer

Step	Operation	Result	Stack After Operation
0	Start	Empty stack	Empty
1	<code>push(7)</code>	Insert 7	7
2	<code>push(9)</code>	Insert 9	7, 9
3	<code>pop()</code>	Returns 9	7
4	<code>push(11)</code>	Insert 11	7, 11
5	<code>stackTop()</code>	Returns 11	7, 11

Final stack:

```
Top -> 11
       7
```

Final result of `stackTop()`:

```
11
```

28. Lesson 1 Checkpoint Questions

Try to answer these without looking at the answers first.

Questions

1. What does LIFO mean?
2. Why can we only insert and delete from the top in a stack?
3. What is the difference between `pop()` and `stackTop()`?
4. What does `top = -1` mean in an array stack?
5. After pushing `10, 16, 20, 25, 30` into an empty stack of size `5`, what is `top`?
6. After one `pop()` from that stack, what value is returned?
7. What does `peek(3)` mean?
8. Why is `push()` usually `O(1)`?
9. What is stack overflow?

10. What is stack underflow?

Answers

1. LIFO means **Last In, First Out**.
 2. Because stack discipline allows access only at the top.
 3. `pop()` removes the top value, but `stackTop()` only reads the top value.
 4. `top = -1` means the stack is empty.
 5. `top = 4`.
 6. The returned value is `30`.
 7. `peek(3)` means read the third value from the top without deleting it.
 8. `push()` is usually `O(1)` because it only changes the top and inserts one value.
 9. Stack overflow happens when we try to push into a full stack.
 10. Stack underflow happens when we try to pop from an empty stack.
-

29. Lesson 1 Key Formulas and Rules

LIFO Rule

Last inserted = first removed

Empty Condition for Array Stack

`top == -1`

Full Condition for Array Stack

`top == size - 1`

Peek Formula for Array Stack

`index = top - position + 1`

Push Effect

top increases by 1
new value is stored at `S[top]`

Pop Effect

```
value at S[top] is saved  
top decreases by 1  
saved value is returned
```

30. Short Glossary

Term	Simple Meaning
Stack	A data structure that follows LIFO
LIFO	Last In, First Out
Top	The position where insertion and deletion happen
Push	Insert a value at the top
Pop	Remove and return the top value
Peek	Read a value from a position without deleting it
stackTop	Read the current top value only
isEmpty	Check if the stack has no values
isFull	Check if the stack has no free space
Overflow	Trying to push into a full stack
Underflow	Trying to pop from an empty stack
ADT	Abstract Data Type; describes operations without focusing on implementation
System stack	Stack used internally for function calls and recursion
Programmer stack	Stack created and controlled by the programmer

31. Final Lesson 1 Summary

A stack is a linear data structure that follows the **LIFO** rule.

LIFO means:

Last In, First Out

The most important position in a stack is the **top**.

All standard stack insertion and deletion happens at the top.

The main stack operations are:

```
push(x)
pop()
peek(pos)
stackTop()
isEmpty()
isFull()
```

In an array stack:

```
top = -1
```

means the stack is empty.

Each successful `push()` increases `top` by `1`.

Each successful `pop()` decreases `top` by `1`.

The basic stack operations are usually `O(1)` because they work directly at the top and do not scan the whole stack.

The stack concept stays the same whether it is implemented using an array, linked list, C structure, or C++ class.

The main idea to remember is:

```
A stack allows access only at the top.
The last value inserted is the first value removed.
```